ETSI TR 104 186 V1.1.1 (2026-10)



TECHNICAL REPORT

Data Solutions (DATA);
Study of the EU Interoperability Test Bed platform

Reference
DTR/DATA-00104186

Keywords
API, data interoperability, NGSI-LD, testing

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed, this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 5
Foreword 5
Modal verbs terminology 5
Executive summary 5
Introduction 6
1 Scope 7
2 References 7
2.1 Normative references 7
2.2 Informative references 7
3 Definition of terms, symbols and abbreviations 7
3.1 Terms 7
3.2 Symbols 8
3.3 Abbreviations 8
4 Study of ITB Platform 8
4.1 Introduction 8
4.2 Analysis 9
4.2.1 Core concepts and organization 9
4.2.2 Test definition and packaging 9
4.2.3 Architecture and deployment 10
4.2.4 Extension mechanism: custom test services 10
4.3 Complete workflow 11
4.3.1 Introduction 11
4.3.2 Scenario set-up 11
4.3.3 Test session execution 11
4.3.4 Output and reporting 11
4.3.5 Adding external test suites 11
5 Integration with the NGSI-LD TTF Robot Framework 11
5.1 Introduction 11
5.2 Integration approach 12
5.2.1 Candidate integration patterns 12
5.2.2 Test structure extraction 12
5.2.3 Selected approach 13
5.3 Prerequisites 13
5.3.1 Introduction 13
5.3.2 ITB instance set-up 13
5.3.3 Test suite conventions 13
5.3.4 Runtime environment 14
5.4 PoC: NGSI-LD TTF connector to the ITB Platform 14
5.4.1 Overview 14
5.4.2 Test generation and deployment (/init) 15
5.4.3 Test execution (/run) 15
5.4.4 Results and limitations 15
5.5 Suggestions and enhancements to the NGSI-LD TTF Robot Framework 16
Annex A: Deployment and configuration details 17
A.1 Docker Compose deployment 17
A.2 Preconfigured community 17
A.3 Test configuration parameters 17
Annex B: Bibliography 18

Annex C: Change history 19
History 20

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Technical Report (TR) has been produced by ETSI Technical Committee Data Solutions (DATA).

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

Executive summary

The present document studies the Interoperability Test Bed (ITB), the conformance testing platform offered by the European Commission, and its applicability to the testing environment of the NGSI-LD API. It analyses the concepts, the architecture and the extension mechanisms of the platform, and reports on a Proof of Concept (PoC) in which the existing NGSI-LD test suite, developed with Robot Framework, is connected to the ITB through an external connector service: ITB test cases are generated automatically from the Robot Framework tests, deployed through the ITB REST API and executed with a native ITB user experience, while the test logic remains solely defined in the Robot Framework test suite. The present document concludes with a set of suggestions making the test suite easier to integrate with external orchestrators such as the ITB.

Introduction

The NGSI-LD API, specified in ETSI GS CIM 009 [i.1], is accompanied by an official test suite [i.2] developed and maintained by the ETSI Testing Task Force (TTF) using Robot Framework. The test suite is the reference tool to assess the conformance of NGSI-LD implementations, and evolves continuously together with the specification.

The Interoperability Test Bed (ITB) of the European Commission offers, on top of executable test suites, the management layer that a conformance programme needs: communities of organizations, conformance statements per system and specification, guided execution with graphical feedback, reporting and certificates. Connecting the NGSI-LD test suite to the ITB would combine the two: the authoritative test logic on one side, and a shared, user-friendly conformance front-end on the other, without duplicating the test suite in another language. The present document studies the ITB platform with this goal in mind and documents a Proof of Concept of such a connection.

1 Scope

The present document studies the EU ITB platform, towards the purpose of directly integrating the current NGSI-LD Test Suite in there. Clause 4 describes the capabilities of the platform, its architecture and the mechanisms it offers to attach external services and test suites. Clause 5 documents a Proof of Concept in which the NGSI-LD test suite, based on Robot Framework, is connected to the ITB without rewriting any test logic, and derives suggestions and enhancements for the test suite itself.

2 References

2.1 Normative references

Normative references are not applicable in the present document.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding, but are not required for conformance to the present document.

[i.1] ETSI GS CIM 009: "Context Information Management (CIM); NGSI-LD API".

[i.2] ETSI Forge: "NGSI-LD API Test Suite repository".

[i.3] European Commission: "Interoperability Test Bed user guide".

[i.4] European Commission: "GITB TDL documentation".

[i.5] European Commission: "GITB test services documentation".

[i.6] Robot Framework Foundation: "Robot Framework User Guide".

[i.7] Robot Framework Foundation: "Robot Framework documentation: Listener interface".

[i.8] European Commission: "Interoperability Test Bed: value proposition".

[i.9] European Commission: "Interoperability Test Bed: Getting started (for developers)".

[i.10] European Commission: "Interoperability Test Bed: User Guide".

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the following terms apply:

actor: role in the information exchanges defined by a specification, played in a test session either by the system under test or by the test bed simulating it

community: set of organizations, users and related configuration managed together in the ITB for a given testing context

conformance statement: association between a system and an actor of a specification, through which the related test cases are executed and the conformance status is tracked

connector service: external service developed in the present study to bridge the ITB and the NGSI-LD Robot Framework test suite

test case: in the ITB, XML document expressed in the GITB Test Description Language defining the steps and assertions executed during a test session

test session: individual execution of a test case against a system, with its configuration, progress and results

test suite: in the ITB, set of test cases packaged with their resources as a ZIP archive and deployed as a whole into a specification

3.2 Symbols

Void.

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
EC	European Commission
GITB	Global eBusiness Interoperability Test Bed
HTTP	HyperText Transfer Protocol
ITB	Interoperability Test Bed
JSON	JavaScript Object Notation
JSON-LD	JavaScript Object Notation for Linked Data
LD	Linked Data
MQTT	Message Queuing Telemetry Transport
MySQL	My Structured Query Language
NGSI	New Generation System Interfaces
PoC	Proof of Concept
REST	REpresentational State Transfer
SHACL	SHApe Constraint Language
SOAP	Simple Object Access Protocol
SUT	System Under Test
TDL	Test Description Language
TTF	Testing Task Force
UI	User Interface
URL	Uniform Resource Locator
WSDL	Web Services Description Language
XML	eXtensible Markup Language

4 Study of ITB Platform

4.1 Introduction

The ITB Platform is a service offered by the European Commission, to support the interoperability and conformance testing of projects that enable the development of cross-border public services.

The platform originates from the Global eBusiness Interoperability Test Bed (GITB) standardization initiative of CEN and is currently maintained and evolved as open-source software by the European Commission (DIGIT), which publishes regular releases [i.3]. It is offered both as a shared instance hosted and operated by the Commission, free to use for projects developing interoperability solutions, and as software that can be installed on premises, its distribution being based on Docker images [i.8].

The ITB presents itself as an online, intuitive and self-service platform for the conformance testing of IT systems against semantic and technical specifications [i.8], covering both the conformance testing of a single component and interoperability testing between components [i.3]. During a test session the test bed drives the exchange of messages with the System Under Test (SUT), simulating the remaining actors foreseen by the specification and validating the content of each exchange.

Clause 4.2 presents the concepts, the architecture and the extension mechanisms of the platform, with attention to the aspects that are relevant when connecting an externally maintained test suite. Clause 4.3 walks through the typical usage workflow, from the initial set-up to the reporting of results.

4.2 Analysis

4.2.1 Core concepts and organization

The ITB organizes testing around a small set of concepts [i.3]:

- the domain is the overall testing context managed by a community, grouping one or more specifications;
- a specification is a standard or profile against which systems are tested; specifications can be organized in groups and expose one or more actors;
- an actor is a role in the information exchanges defined by the specification; in each test case one actor is played by the SUT, while the other actors are simulated by the test bed;
- test suites and test cases are the executable artefacts, expressed in the GITB TDL [i.4], that verify a specification (see clause 4.2.2);
- communities, organizations and systems provide the administrative grouping: a community gathers the organizations participating in a testing context, and each organization registers the systems that implement the specification;
- a conformance statement associates a system with a specification actor; it is the entry point for executing the related test cases and for tracking the conformance status over time.

This organization decouples the definition of the tests (managed by the community administrators) from their execution (performed by the organizations on their own systems), which is what enables a conformance programme to be operated as a service. Administration follows the same hierarchy, with distinct test bed, community and organization administrator roles [i.10].

4.2.2 Test definition and packaging

Tests are described in the GITB Test Description Language (TDL) [i.4], an XML language in which a test case declares its metadata, the involved actors and a sequence of steps. Steps cover, among others, messaging between actors (send, receive), verification of content (verify), data manipulation and utility processing (assign, process), user interactions (interact), flow control (if, while, group) and the computation of the final verdict with dedicated success and failure messages. Reusable sequences of steps can be factored into scriptlets, and each session maintains its state as a context of key-value pairs [i.9].

Test cases are packaged into test suites: ZIP archives containing a test suite descriptor (testSuite.xml), the test case definitions and any resource files needed at execution time (templates, schemas, example payloads). Test suites are deployed into a specification either through the user interface or through the REST API (see clause 4.2.3). Uploaded test suites are validated in depth by the test bed, and a standalone validator for TDL documents is also provided [i.9].

The TDL offers built-in capabilities for common needs, including messaging over widespread protocols (e.g. REST, SOAP, AS2, AS4), validation of content against standard specification languages (e.g. XML Schema, Schematron, SHACL), template processing and content display, so that many scenarios are covered without any custom development [i.8]. Beyond the built-in capabilities, a step can delegate its processing to an external service (see clause 4.2.4).

Values can be drawn at runtime from the configuration of the involved entities, through expressions that reference properties of the domain, the specification, the organization or the system being tested (e.g. $SYSTEM{endpointAddress}). This mechanism decouples the test logic from deployment-specific parameters, which are maintained in the test bed rather than in the test cases.

4.2.3 Architecture and deployment

The platform is composed of four cooperating services distributed as Docker images [i.3]: a web frontend (gitb-ui) serving the user interface and the REST API; the test engine (gitb-srv) executing the test sessions; a MySQL database for persistence; and a Redis cache for user session management.

A deployment can rely on the shared instance operated by the European Commission or be self-hosted, typically through a Docker Compose description that brings up the complete platform on a single machine; the Docker images are publicly available, including nightly builds giving early access to upcoming features [i.9]. The functional capabilities are the same in both cases.

The frontend exposes a REST API for automation purposes, disabled by default. It covers, among others, the deployment of test suites, the management of communities, organizations and systems, and the launch of test sessions. Calls are authenticated by means of API keys: a key identifying the community (or the test bed administrator) is carried in a dedicated HTTP header, while the payload identifies the target resources (e.g. the specification onto which a test suite is deployed) through their own keys [i.3].

An instance can also be initialized automatically at start-up by importing a data archive containing a complete configuration (community, specifications, test suites, users and API keys). This data import mechanism makes it possible to distribute a fully preconfigured test bed, as done by the PoC described in clause 5. Conversely, the configuration of an instance can be exported as an archive, e.g. to align a production instance with a development one; administrators can further monitor the health of the platform services [i.10].

4.2.4 Extension mechanism: custom test services

The main extension point of the ITB consists of custom test services: external web services, implemented and hosted independently of the test bed, that TDL steps invoke at execution time [i.5]. Three service types are defined:

- validation services, used by verify steps to implement custom validation logic;
- messaging services, used by send and receive steps to implement custom communication protocols;
- processing services, used by process steps to implement arbitrary operations and data transformations.

The contract between the test bed and a custom service is defined by the GITB service APIs: SOAP web services described by a WSDL, with a small set of operations (e.g. getModuleDefinition, process, and transaction-oriented variants for stateful exchanges) and a generic, typed structure for inputs and outputs. A TDL step designates the service to be used simply by referencing the URL of its WSDL in the handler attribute of the step. Services can be implemented in any technology; a template project (a Maven archetype based on Java and the Spring framework) is provided to bootstrap implementations [i.5], [i.9].

Custom test services are deployed and operated independently of the test bed, on the infrastructure of the community that provides them; hosting by the Commission can also be considered for eligible projects [i.8]. A common practice is to record the address of the services in domain-level configuration parameters, keeping the test cases portable across environments [i.9].

Through this mechanism the test bed can execute built-in and externally provided test suites [i.8].

4.3 Complete workflow

4.3.1 Introduction

An example of a running workflow that highlights the typical usage of the ITB Platform is presented in the following clauses, from the initial set-up (clause 4.3.2) to the execution of test sessions (clause 4.3.3), the evaluation of the produced output (clause 4.3.4) and the addition of external test suites (clause 4.3.5).

4.3.2 Scenario set-up

The set-up of a testing scenario is an administrative activity performed once [i.3]. A community administrator defines the domain, the specifications and their actors, and deploys the corresponding test suites. Organizations are registered in the community, each declaring the systems that implement the specification.

A conformance statement is then created for a system, selecting the specification and the actor played by the SUT. Configuration properties are provided at this point (for instance the endpoint address of the system): they parameterize the test cases as described in clause 4.2.2.

4.3.3 Test session execution

From a conformance statement, the user reaches the list of applicable test cases and launches one, several or all of them. The test engine executes the TDL steps, driving the SUT and the simulated actors. The execution is followed live in the browser as a sequence diagram between the actors, each step being marked as succeeded, failed or skipped, with the session log accumulating alongside.

Test cases can pause for user interactions when a manual action or input is required. Test sessions can also be launched non-interactively through the REST API (see clause 4.2.3), which is of interest for automation in CI/CD pipelines.

4.3.4 Output and reporting

Each step exposes a report with the exchanged or validated content, and the session ends with an overall verdict (success, failure or undefined). Reports can be exported at step, session and conformance statement level. The conformance statement view aggregates the results over time, showing which test cases have been passed; when the community enables it, a conformance certificate can be produced once the statement is fulfilled. This user experience, in which the detail of every exchange is inspectable in place, is a substantial improvement over browsing raw log files, and is the main motivation for the integration studied in clause 5.

4.3.5 Adding external test suites

A test suite packaged as described in clause 4.2.2 is added to a specification through the user interface (upload of the ZIP archive, see [i.10]) or programmatically through the REST API, the latter enabling a re-deployment to be triggered automatically whenever the test suite evolves.

Combined with the data import mechanism (see clause 4.2.3), this allows the full lifecycle to be automated: an initially empty instance can be preconfigured at start-up and continuously updated with re-generated test suites, which is the operating model of the PoC described in clause 5.

5 Integration with the NGSI-LD TTF Robot Framework

5.1 Introduction

The objective of the task is a study of the potential addition of the ITB Platform in the NGSI-LD testing environment. As of today, the test suite of the NGSI-LD standard [i.1] is developed and maintained using Robot Framework [i.6]. It is not an objective to replace Robot Framework by the TDL (Test Description Language) used in ITB, nor to develop in parallel a full test suite in TDL that would be a copy of it.

From here, the proposal was to study the possibility to use the existing Robot Framework test suite in the ITB environment, and to think about the management of this integration considering its continuous evolution.

Consequently, the possibility of finding a way to call the test suite was studied. The idea is to generate, for each Robot Framework test, an ITB test that calls and displays the results of this specific Robot Framework test. Thanks to this process, the evolution of the Robot Framework test suite remains independent of ITB: the test suite can be modified, and only the regeneration of the ITB tests is then required.

The PoC presented in the following clauses aims to demonstrate the feasibility and advantages of such an approach.

This PoC should demonstrate that the simplest test cases can be handled generically, and that the most complex ones (specific actions) will require further iterations.

Clause 5.2 discusses the candidate integration approaches, built on the platform capabilities described in clause 4, and motivates the selected one. Clause 5.3 collects the prerequisites of the integration, clause 5.4 reports on the PoC itself and clause 5.5 gathers the resulting suggestions for the test suite.

5.2 Integration approach

5.2.1 Candidate integration patterns

The test cases executed by the test bed are expressed in the GITB TDL (see clause 4.2.2), while the NGSI-LD test suite [i.2] is maintained in Robot Framework [i.6]. The integration therefore relies on the extension mechanism described in clause 4.2.4: one or more custom test services, implemented and hosted alongside the test suite, that the TDL steps invoke at execution time. Since the test engine and the custom services interact over web service calls, the practical constraint is network reachability between the two (and towards the SUT), rather than the installation of any component inside the test bed.

A first candidate pattern is coarse-grained: a single, monolithic, TDL test case delegates a whole run of the Robot Framework test suite to a custom service. The user selects, through configuration properties, which part of the test suite to execute; the service runs it and returns the native reports, which the test bed displays or offers for download. This pattern is inexpensive to set up and preserves the reports the NGSI-LD community is used to; on the other hand, a test session collapses into essentially one step, and the verdict refers to an entire run rather than to individual tests.

A second candidate pattern is fine-grained: one TDL test case is provided for each test of the Robot Framework test suite. At execution time the generated test case calls a processing service that runs the corresponding Robot Framework test and returns its steps, together with their inputs, outputs and statuses; the TDL steps then replay this information in the test bed. This yields one conformance item per test, a sequence diagram per session and per-step verdicts, while the test logic remains solely defined in the Robot Framework test suite. The cost is a generation toolchain, and a number of conventions on the test suite so that its structure can be extracted automatically.

Both patterns can be complemented by direct TDL authoring: particular test cases (e.g. long-running asynchronous scenarios) can be implemented natively in TDL, alongside the generated ones, whenever their automatic generation is not cost-effective.

5.2.2 Test structure extraction

The fine-grained pattern requires extracting from the test suite the structure of every test: its phases, its significant steps and the actors involved. Three options were considered:

- parsing the Robot Framework source code, which reflects the definition of the tests but requires re-implementing the resolution logic of the framework;
- parsing the execution output of the framework (output.xml, also available as JSON in recent Robot Framework versions [i.6]), which is the richest source but unrolls loops and embeds a level of detail beyond the visual steps to be shown;
- attaching a listener [i.7] to a dry-run execution, in which Robot Framework resolves and traverses the keywords without executing their implementation: the listener observes the exact keyword invocations and can retain only those that are meaningful as visual steps.

The listener over a dry-run was selected: it lets the framework itself resolve the structure of the tests, keeps the extraction logic small (a filter over keyword names and tags), and requires no access to a running SUT.

5.2.3 Selected approach

The retained integration follows the fine-grained pattern, with the structure extracted through listeners over a dry-run. The generated test cases rely on two TDL step types only: messaging steps, rendering the recorded exchanges between the actors and the SUT, and processing steps, invoking the connector service and rendering the recorded verifications. The results are injected into this fixed structure when the test is executed, in a static replay.

An event-based alternative, in which the steps would be reported to the test bed while the Robot Framework test is executing, was also analysed. The static replay was retained as the more universal option, since it applies also to tests whose execution cannot be split into separate events; the event-based bridge is identified as a possible evolution of the PoC.

The selected approach keeps the Robot Framework test suite as the single source of truth, produces a native test bed experience for the end user and is largely generic: it can in principle be reused for other Robot Framework based test suites, given the list of keywords and tags that the listeners recognize.

5.3 Prerequisites

5.3.1 Introduction

The present clause collects the prerequisites for attaching the current NGSI-LD TTF Robot Framework test suite to the ITB Platform, in order to leverage the graphical representation and the user experience of the test bed when executing the test suite. They concern the ITB instance (clause 5.3.2), the test suite itself (clause 5.3.3) and the runtime environment (clause 5.3.4).

5.3.2 ITB instance set-up

The PoC relies on a self-hosted ITB instance, deployed locally with Docker Compose together with the connector service described in clause 5.4 (see annex A). The REST API of the instance is enabled, and the instance is initialized at start-up through the data import mechanism (see clause 4.2.3) with a preconfigured community: the testing domain, the NGSI-LD specification with its actors, a placeholder system with its conformance statements, and the API keys used by the connector to deploy the generated test suites.

Nothing in the approach is specific to a local deployment: the same integration can target an instance hosted by the European Commission. In that case the API keys are obtained from the administration of the community on that instance, and network reachability becomes the main prerequisite: the ITB test engine invokes the connector service, while the connector invokes the ITB REST API and reaches the SUT. In the local deployment of the PoC, all components share a Docker network to which the context broker under test is attached as well. In the longer term, an instance operated by ETSI for the NGSI-LD community is one of the deployment options considered.

5.3.3 Test suite conventions

The generation of the ITB test cases does not require any modification of the test logic, yet it relies on a number of conventions that the NGSI-LD test suite [i.2] follows:

- the keywords representing observable interactions or verifications follow recognizable naming patterns: the HTTP verbs for the requests towards the SUT, names containing "Check" for the verifications, "Wait for ..." for the synchronization points;
- the interaction keywords carry tags identifying the actor performing them (e.g. actor_context-consumer, actor_notification-receiver), so that each step can be attributed to the right participant in the ITB sequence diagram;
- every test can be executed in Robot Framework dry-run mode [i.6], so that its structure (set-up, steps, teardown) can be enumerated without a running SUT;

- all deployment-specific inputs (endpoints, hosts, ports, @context URLs) are externalised as Robot Framework variables that can be injected at launch time (see the mapping in annex A).

NOTE: The actor tags were introduced in the test suite in the context of the present study; clause 5.5 elaborates on this and on the other conventions.

5.3.4 Runtime environment

The complete environment consists of the ITB containers, the connector service and the SUT, all reachable within the same network. The connector exposes an HTTP endpoint for the generation phase and the GITB processing service consumed at execution time. The SUT is an NGSI-LD context broker [i.1], accompanied where needed by the auxiliary components that some test groups expect (e.g. an MQTT broker or a context source). Annex A details the Docker Compose deployment and the full list of configuration parameters.

5.4 PoC: NGSI-LD TTF connector to the ITB Platform

5.4.1 Overview

This PoC focuses on the development of an external ITB service (the connector service, available in the test suite repository [i.2]), which implements the integration approach selected in clause 5.2 and is assigned two distinct tasks:

- an initialization operation (/init), which generates ITB test cases from the Robot Framework tests and deploys them on the ITB platform (see clause 5.4.2);
- a GITB processing service (see clause 4.2.4), called by the first operation of every generated ITB test case, which launches the corresponding Robot Framework test and returns the test results to ITB, step by step (see clause 5.4.3).

The test inputs (such as the SUT endpoints and related parameters) are not known to the connector service: they are provided by the user in ITB, as configuration properties of the system under test, and reach the connector within each call of the processing service.

Figure 1 depicts the resulting architecture: the initialization flow (steps labelled 1.x, see clause 5.4.2) and the execution flow (steps labelled 2.1, 2.2, 2.3 and 2.4, see clause 5.4.3).

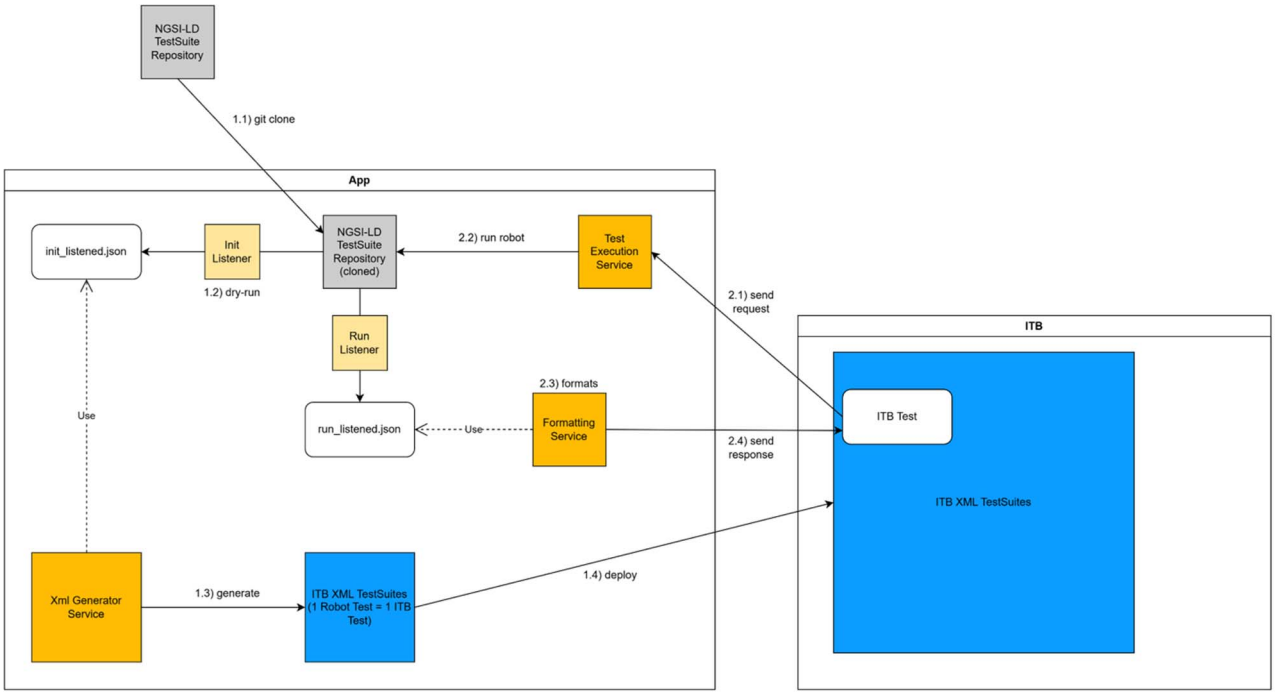


Figure 1: Architecture and workflow of the connector service

5.4.2 Test generation and deployment (/init)

The initialization, corresponding to the steps labelled 1.1, 1.2, 1.3 and 1.4, in Figure 1, proceeds in four steps:

- **Git clone:** a copy of the NGSI-LD Test Suite repository [i.2] is created in the connector service. The repository URL is configurable, so that a fork can be used in place of the official test suite.
- **Dry-run:** the service dry-runs the Robot Framework test suite [i.6] with a dedicated listener [i.7] attached. The listener creates a JSON report capturing, for every test, its set-up, test and teardown phases, the main keywords of each phase (corresponding to the test steps that ITB displays) and the actors involved, derived from the keyword tags (see clause 5.3.3).
- **ITB XML generation:** the service extracts from the JSON report the test suites and the test cases they contain, and generates one ITB test case per Robot Framework test, assembling TDL fragments from templates. The actors declared in each test case are those observed during the dry-run; each HTTP keyword becomes a messaging step between its actor and the SUT, and each verification keyword becomes a step displaying the recorded outcome. The generated test cases are grouped into one ITB test suite per thematic area of the Robot Framework test suite, each completed with the mandatory testSuite.xml descriptor and with a resource template (test-request.json) used at runtime to assemble the test inputs.
- **Deployment in ITB:** each generated test suite is packaged as a ZIP archive (see [i.10]) and deployed through the ITB REST API, authenticated with the preconfigured API keys (see clause 5.3.2).

Within each generated test suite the test cases are ordered alphabetically by identifier, and the Robot Framework environment of the connector is aligned with the test suite through the requirements file of the repository. Since the generation starts from a plain checkout of the test suite, the initialization can be re-invoked whenever tests are added or modified, refreshing the deployed test suites without manual intervention. Introducing this invocation in the CI/CD pipeline of the test suite would keep the two automatically aligned.

5.4.3 Test execution (/run)

When a generated test case is launched in ITB (steps labelled 2.1, 2.2, 2.3 and 2.4 in Figure 1), its first step assembles the input payload (the test name, together with the parameters taken from the configuration of the system and of the domain) and calls the processing service of the connector. The service runs the corresponding Robot Framework test, this time against the configured SUT, with a second listener attached: it records, for every significant keyword, its arguments, the payloads used (e.g. the JSON bodies loaded from the test suite resources) and its status.

The recorded information is returned to the test bed as a structured map, and the subsequent steps of the generated test case replay the execution: each recorded request appears as a message from its actor to the SUT, and each verification as a processing step, both exposing the recorded inputs and outputs together with the individual pass or fail status. The verdict of the session is derived from the statuses of the replayed steps. Dedicated failure messages distinguish an unreachable connector service and a failed set-up (in which case the test steps are skipped) from a genuine test failure.

The presentation in the test bed was refined in the course of the study: the internal preparation of the session is grouped under the set-up phase, each step is attributed to the NGSI-LD actor performing it rather than to the test engine, and the configuration exposed to the end user is reduced to the essential properties (in the typical case, the endpoint of the context broker under test), the remaining parameters being maintained at domain level (see annex A).

5.4.4 Results and limitations

The PoC demonstrates the feasibility of the approach end to end: generation over the full test suite, deployment through the REST API, and execution with a native rendering in ITB, following the workflow of Figure 1, without any duplication of the test logic. As anticipated in clause 5.1, the simplest test cases are handled generically, while the most complex ones require further iterations.

The current limits of the connector service, which delimit those iterations, are the following:

- only the keywords matching the naming conventions of clause 5.3.3 are captured by the listeners; steps expressed differently, including the synchronization keywords of the asynchronous subscription and notification scenarios (e.g. "Wait for notification"), do not surface in the generated test cases yet;

- the attribution of steps to actors depends on the presence of the actor tags; untagged keywords fall back to a default actor;
- the payload capture covers the inputs loaded from the test suite resources and the recorded requests and responses, but not every intermediate artefact a test may produce;
- the grouping of the generated test suites mirrors the folder structure of the Robot Framework test suite; a different organization in ITB (e.g. by specification clause) would call for an explicit mapping.

Two complementary directions were identified during the study: for particular cases whose automatic generation is not cost-effective, the direct TDL authoring mentioned in clause 5.2.1 remains applicable; and an ongoing verification compares the results reported by the test bed with the native Robot Framework output, so as to confirm that the two remain aligned. The event-based bridge discussed in clause 5.2.3 is the main evolution identified for a further iteration of the PoC.

These limitations concern the connector service rather than the approach itself, and they translate into the suggestions on the test suite collected in clause 5.5.

5.5 Suggestions and enhancements to the NGSI-LD TTF Robot Framework

The analysis of the platform in clause 4 and the limitations found in clause 5.4 lead to a set of suggestions on the NGSI-LD TTF Robot Framework test suite. None of them changes the test logic: they aim at making the test suite easier to import into the ITB as an external test suite (and, more generally, into any external orchestrator), by removing the fragilities observed during the PoC:

- **Systematic actor tagging:** tags identifying the acting role (e.g. actor_context-consumer, actor_notification-receiver) were added to the interaction keywords of the test suite in the context of the present study. Applying them systematically, including to newly added tests, is what keeps the generated sequence diagrams faithful.
- **Stable keyword naming conventions:** the extraction currently recognizes the interaction and verification keywords through naming patterns (the HTTP verbs, names containing "Check", "Wait for ..."). Documenting these conventions in the test suite, or complementing them with dedicated keyword tags, would turn an implicit dependency into an explicit contract.
- **Stable and unique test identifiers:** the identifiers of the generated ITB test cases derive from the Robot Framework test names. Keeping the test names unique across the test suite and stable over time preserves the conformance history in ITB across regenerations.
- **Dry-run friendliness:** the structure extraction relies on the Robot Framework dry-run mode. Avoiding constructs that hide steps from a dry-run (e.g. dynamically composed keyword names) keeps the generated test cases complete.
- **Consolidated configuration catalogue:** the test suite exposes its deployment-specific inputs as externally injectable variables. Maintaining them as a single documented catalogue, aligned with the ITB configuration properties (see annex A), simplifies the mapping and its evolution.
- **Keyword documentation:** the documentation of the significant keywords can be surfaced by the generated test cases as step descriptions. Keeping it short and meaningful improves the readability of the test sessions in ITB.
- **Co-located extraction listeners:** hosting the extraction listeners [i.7] in the test suite repository itself, so that their keyword-recognition logic evolves together with the test suite, would avoid a divergence between the two.

No blocking issue attributable to the test suite itself was identified during the PoC: the suggestions above are incremental hardenings of conventions that the test suite already largely follows.

Annex A:
Deployment and configuration details

A.1 Docker Compose deployment

The PoC environment is described by a Docker Compose file, available in the itb folder of the test suite repository [i.2]. It starts the four ITB containers (gitb-ui, gitb-srv, gitb-mysql, gitb-redis) together with the connector service (robot-processing). All containers join a shared Docker network, to which the context broker under test is attached as well. The ITB user interface is served on the local port 9000 and the connector service listens on the port 8080.

The connector image embeds the Robot Framework runtime and the extraction listeners; at initialization time the connector clones the test suite repository as described in clause 5.4.2.

A.2 Preconfigured community

At the first start-up, the ITB instance automatically imports a configuration archive mounted in its data directory, using the data import mechanism described in clause 4.2.3. The archive creates a dedicated community with the NGSI-LD specification and its actors, a sample system with the related conformance statements, and the community and specification API keys used by the connector service for the deployment of the generated test suites. The REST API of the instance is enabled through the instance configuration. The default credentials created by the import are expected to be changed at the first login.

A.3 Test configuration parameters

Table A.1 lists the configuration properties defined in ITB and the test suite variables they feed, as assembled by the generated test cases at execution time (see clause 5.4.3). The first two properties are configured per system, as they identify the SUT; the remaining ones are defined at domain level and shared by the community.

Table A.1: Mapping between ITB configuration properties and test suite variables

ITB configuration property	Test suite variable	Description
endpointAddress	url	Base URL of the NGSI-LD API of the SUT (the ngsi-ld/v1 path is appended by the generated test cases)
temporalApiUrl	temporal_api_url	Base URL of the temporal API of the SUT, when exposed separately
ngsildTestSuiteContext	ngsild_test_suite_context	URL of the JSON-LD @context file used by the test suite
notificationServerHost	notification_server_host	Host on which the test suite listens for notifications
notificationServerPort	notification_server_port	Port on which the test suite listens for notifications
mqttBrokerHost	mqtt_broker_host	Host of the MQTT broker used by the MQTT notification tests
mqttBrokerNonDefaultPort	mqtt_broker_non_default_port	Non-default port of the MQTT broker
contextSourceHost	context_source_host	Host of the context source simulated by the test suite
contextSourcePort	context_source_port	Port of the context source simulated by the test suite
contextServerHost	context_server_host	Host of the server distributing JSON-LD @context files during the tests
contextServerPort	context_server_port	Port of the server distributing JSON-LD @context files during the tests
coreContext	core_context	URL of the NGSI-LD core @context [i.1]
deleteTemporalOnCoreDelete	delete_temporal_on_core_delete	Indication of whether the deletion of an entity also deletes its temporal representation

Annex B: Bibliography

- Portal regrouping all the ITB documentation.

Annex C:
Change history

Date	Version	Information about changes
November 2025	0.0.1	Early draft
February 2026	0.1.1	Stable draft
July 2026	0.1.2	First complete draft: clauses 2.2, 3.1 and 3.3 completed; clauses 4 and 5 populated (study of the ITB platform; PoC connecting the NGSI-LD Robot Framework test suite); annex A added
July 2026	0.1.3	Clause 4 aligned with the official ITB guides (references [i.8] to [i.10]); integration considerations moved from clause 4 into the new clause 5.2; architecture figure relocated to the PoC overview; clause 5 enriched with the outcomes of the ITB-ETSI-DIGIT coordination meetings

History

Version	Date	Status
V1.1.1	October 2026	Publication